



第3章:SQL简介

数据库系统概念, 6th编辑。

©Silberschatz, Korth和Sudarshan参见www.db-book.com了解
重用条件



第3章:SQL简介

“SQL查询语言概述” “数据定义

基本查询结构

附加基本操作

“集合操作”

“空值”

“聚合函数”

“嵌套子查询”

数据库的修改



历史

IBM Sequel语言是在IBM圣何塞研究实验室开发的System R项目的一部分

重新命名的结构化查询语言(SQL)

ANSI和ISO标准SQL:

SQL-86

SQL-89

SQL-92

z SQL:1999(语言名称符合Y2K !)

SQL:2003

商业系统提供了大部分(如果不是全部的话)SQL-92特性, 以及来自后来的标准和特殊专有特性的不同特性集。

z并非这里的所有示例都适用于您的特定系统。



SQL中的域类型

- ☒ **char (n)**。固定长度的字符串，用户指定长度为 n 。
 - ☒ **varchar (n)**。可变长度字符串，具有用户指定的最大长度 n 。
 - ☒ **int**。整数(与机器相关的整数的有限子集)。
 - ☒ **短整型**。小整数(整数域类型的与机器相关的子集)。
 - ☒ **数字(p d)**。定点数，用户指定精度为 p 位，小数点右边有 n 位。
 - * **实数，双精度**。浮点数和双精度浮点数，具有机器依赖的精度。
 - ☒ **漂浮(n)**。浮点数，用户指定的精度至少为 n 位。
- 更多内容将在第4章中介绍。



创建表构造

安SQL关系是用**create table**命令定义的:

```
create table  $r$  ( $A_1 D_1, A_2 D_2, \dots$ ), 安,  
    (integrity-constraint1),  
  
    ...,  
    (integrity-constraintk))
```

r 是关系的名称

A_i 是每个关系 r 模式中的一个属性名

D_i 是属性 A_i 域中值的数据类型示例:

创建表教员(

ID **char** (5),

Name **varchar**(20) **not null**, *dept_*

name **varchar**(20),

2)工资数字(8日)

```
' insert into instructor values (' 10211 ', ' Smith ', ' Biology ',  
66000);' insert into instructor values (' 10211 ', null, ' Biology ',  
66000);
```



I创建表的完整性约束

* **not null**

“主键($A_1, \dots, A_n$)

“外键($A_m, \dots, A_n$)引用 r

示例:声明*branch_name*为分支的主键

创建表*instructor* ()

ID **char** (5),

Name **varchar**(20) **not null**, *dept_*

name **varchar**(20),

薪水数字(8,2),

主键(ID),

外键(*dept_name*)引用*department*)

属性上的主键声明自动确保不为空



以及更多的关系定义

```
“create table student (  
    ID varchar(5)主键,  
    Name varchar(20)不为空,  
    dept_name varchar (20),  
    tot_cred数字(0)  
    外键(dept_name)引用department));
```

```
“create table takes ()  
    ID varchar(5) 主 键 , course_id varchar  
    (8)  
    Sec_id varchar(8), 学 期  
    varchar(6),  
    Year number (4, 0), grade  
    varchar(2),  
    外键(ID)引用student,  
    外键(course_id, sec_id, semester, year)引用section);
```



还有更多

- **create table** *course* (
 Course_id **varchar**(8)主键,
 标题**varchar** (50),
 dept_name **varchar** (20),
 信用数字(2,0),
 外键(*dept_name*)引用*department*));



删除和修改表结构

“删除表”
修改表

b. 修改表

其中 A 是要添加到关系 r 中的属性的名称, D 是 A 的域。

关系中的所有元组被赋为 $null$ 作为新属性的值。

z **alter table r drop A**

其中 A 是关系 r 的属性名

删除许多数据库不支持的属性。



基本查询结构

一个典型的SQL查询有这样的形式:

```
select  $A_1, A_2, \dots, A_n$   
from  $r_1, r_2, \dots, r_m$   
where  $P$ 
```

A_i 表示一个属性

R_i 表示一个关系

P 是一个谓词。

SQL查询的结果是一个关系。



select子句

select子句列出查询结果中所需的属性

z对应关系代数的投影运算。例:求所有教员的名字:

选择的名字

从教练

注:SQL名称不区分大小写(即, 您可以使用大写或小写字母)。

z例如, *Name* $\equiv$ Name $\equiv$ Name

有些人我们在用粗体的地方用大写。



select子句(接续)

SQL允许在关系和查询结果中重复。

要强制消除重复项，请在select后插入关键字**distinct**。

*查找所有带讲师的院系名称，并删除重复项

选择不同的dept_name
从教练

☒ 关键字**all**指定不删除重复项。

从教员中选择所有dept_name



select子句(接续)

select子句中的星号表示“所有属性”。

```
Select * from  
m instructor
```

“select子句可以包含算术表达式，包括操作+、-、*和/，以及对元组的常量或属性进行操作。

◆查询:

```
select ID, name, salary/12  
from instructor
```

将返回一个与教员关系相同的关系，只是属性salary的值除以12。



where子句

where子句指定结果必须满足的条件

z对应于关系代数的选择谓词。**寻找**比较科学领域的所有讲师。年薪80000的部门

选择的**名字**

从**教练**

where *dept_name* = ' 比较。科学。 ,工资8万

比较结果可以使用逻辑连接词**and**, **or**, and **not**进行组合。

比较可以应用于算术表达式的结果。



from子句

from子句列出查询中涉及的关系

z对应于关系代数的笛卡儿积运算。

找到x教授的笛卡儿产品讲师

选择*

来自讲师,教导

z生成所有可能的教员-教员对，所有属性都来自这两个关系。

笛卡儿积不是很直接有用，但与where子句条件(关系代数中的选择运算)结合使用时很有用。



Cartesian Product

教练

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000

教

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009

Inst.ID	name	dept_name	salary	teaches.ID	course_id	sec_id	semester	year
10101	Srinivasan	Physics	95000	10101	CS-101	1	Fall	2009
10101	Srinivasan	Physics	95000	10101	CS-315	1	Spring	2010
10101	Srinivasan	Physics	95000	10101	CS-347	1	Fall	2009
10101	Srinivasan	Physics	95000	10101	FIN-201	1	Spring	2010
10101	Srinivasan	Physics	95000	15151	MU-199	1	Spring	2010
10101	Srinivasan	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Physics	95000	10101	CS-101	1	Fall	2009
12121	Wu	Physics	95000	10101	CS-315	1	Spring	2010
12121	Wu	Physics	95000	10101	CS-347	1	Fall	2009
12121	Wu	Physics	95000	10101	FIN-201	1	Spring	2010
12121	Wu	Physics	95000	15151	MU-199	1	Spring	2010
12121	Wu	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...



连接

对于所有教过课程的讲师，找到他们的名字和所教课程的课程ID。

选择 *name, course_id*
来自 *讲师, 教课*
教练的 **地方**。ID = 教官。ID

查找 Comp Sci 提供的每门课程的课程编号、学期、学年和名称。部门

选择 *部分。Course_id, 学期, 学年, 标题*
来自 *section, course*
部分的 **地方**。Course_id = 课程。course_id 和 dept_
name = 'Comp。Sci。’



试着用SQL写一些查询

*建议编写查询.....



Natural Join

“自然连接对所有公共属性匹配具有相同值的元组，并且只保留每个公共列的一个副本



select *

从教官自然加盟教;

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Comp. Sci.	65000	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	CS-347	1	Fall	2009
12121	Wu	Finance	90000	FIN-201	1	Spring	2010
15151	Mozart	Music	40000	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	PHY-101	1	Fall	2009
32343	El Said	History	60000	HIS-351	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-319	1	Spring	2010
76766	Crick	Biology	72000	BIO-101	1	Summer	2009
76766	Crick	Biology	72000	BIO-301	1	Summer	2010



Natural Join(续)

“自然连接的危险:当心同名的不相关属性被错误地等同。

列出教师的姓名以及他们所教的课程名称

不正确的版本(将course.dept_name等同于instructor.dept_name) **z** `select name,
title`

From instructor natural join教授**natural join**课程;

“正确版”

z选择姓名、标题

从讲师自然加盟教、课

教的地方。`course_id = course.course_id;`

“另一个正确版本”

z选择姓名、标题

From (instructor natural join teachings) join course using(course_id);



The Rename Operation

SQL允许使用**as**子句重命名关系和属性:

旧名作为新名

☒如

z **select** *ID, name, salary/12 as monthly_salary*
从教练

“找出所有薪水比较公司中某些教员高的教员的名字。” 科学”。

z **select distinct** *T. name*
from *instructor as T, instructor as S*
where *T.salary > S.salary and S.dept_name = '比较'。科学”。*

“关键字**as**是可选的，可以省略讲师**as T**≡讲师**T**



字符串操作

SQL 包含一个字符串匹配操作符，用于对字符串进行比较。操作符“like”使用使用两个特殊字符描述的模式：

z %(%). %字符匹配任何子字符串。

z 下划线(_). _字符匹配任何字符。

*查找姓名中包含子字符串“dar”的所有教员的名字。

选择的名字
从教练
名字像'%dar%'

匹配字符串“100%”

比如' 100 \% ' escape '\'

SQL 支持多种字符串操作，如z级连接(使用" || ")

z 从大写转换为小写(反之亦然)z 查找字符串长度，提取子字符串等。



对元组的显示排序

按字母顺序列出所有教官的名字，选择不同的名字

从教练

按姓名排序

对于每个属性，我们可以指定**desc**为降序或**asc**为升序;升序为默认值。**z**示例:按名称**desc**排序

可以对多个属性进行排序

z示例:按dept_name, name .排序



Where子句谓语句

SQL包含一个**Where**比较操作符

“示例:查找工资在\$90,000到\$100,000之间(即 $\geq \$90,000$ 和 $\leq \$100,000$)的所有
教员的姓名

选择姓名

从教练

工资在90000到100000之间的哪里

“元组比较”

选择名称, *course_id*

From *instructor*, 教课

(讲师。ID, *dept_name*) =(教官。ID、 “生物学”);



重复的

- ☒ 在有重复项的关系中，SQL可以定义结果中出现多少个元组的副本。
- ☒ 一些关系代数运算符的**多集**版本——给定多集关系 r_1 和 r_2 :
 1. $\sigma_{\theta}(r_1)$:如果元组 t_1 在 r_1 中有 c_1 个副本，且 t_1 满足选择 σ_{θ} ，则在 $\sigma_{\theta}(r_1)$ 中有 c_1 个 t_1 副本。
 2. $\Pi_A(r)$:每个元组 t_1 在 r 中有 c_1 个副本，有 c_1 份tuple $\Pi(t_1)$ ， $\Pi(t_1)$ 表示单一元组 t_1 的投影。
 3. $r_1 \times r_2$:如果在 r_1 中有 c_1 个副本的tuple t_1 和 c_2 个副本的tuple t_2 ，则有 $c_1 \times c_2$ 个元组 T_1 的副本 T_2 在 $r_1 \times r_2$ 中。



副本(续)。

示例:假设多集关系 $r_1(A, B)$ 和 $r_2(C)$ 如下:

$$R_1 = \{(1, a) (2, a)\} \quad R_2 = \{(2), (3), (3)\}$$

那么 $\Pi_B(r_1)$ 将是 $\{(a), (a)\}$, 而 $\Pi_B(r_1) \times r_2$ 将是

$$\{(a, 2), (a, 2), (a, 3), (a, 3), (a, 3), (a, 3), (a, 3)\}$$

SQL重复语义:

```
select  $A_1, A_2, \dots, A_n$   
from  $r_1, r_2, \dots, r_m$   
where  $P$ 
```

相当于表达式的多集版本:

$$\Pi_{A_1, A_2, \dots, A_n} (\sigma_P(r_1 \times r_2 \times \dots \times r_m))$$



集合操作

查找2009年秋季或2010年春季的课程(*select course_id from section where sem = ' Fall ' and year = 2009*)联合

(*select course_id from section where sem = ' Spring ' and year = 2010*)

“查找2009年秋季和2010年春季的课程(*select course_id from section where sem = ' Fall ' and year = 2009*)相交。

(*select course_id from section where sem = ' Spring ' and year = 2010*)

“查找2009年秋季运行但不是2010年春季运行的课程(*select course_id from section where sem = ' Fall ' and year = 2009*)除非

(*select course_id from section where sem = ' Spring ' and year = 2010*)



集合操作

*设置操作**union**、**intersect**和**except**

z 以上每个操作都会自动消除重复项。要保留所有重复项，请使用相应的多集版本**union all**, **intersect all**和**except all**。

“假设一个元组在 r 中出现 m 次，在 s 中出现 n 次，那么，它出现：
z $m + n$ 次在 r 中出现**union all** s

z $\min(m, n)$ 次在 r 中与**所有** s 相交

z $\max(0, m - n)$ 次在 r 中**除所有** s



空值

对于元组的一些属性，可能有一个空值，用`null`表示

“`null`”表示一个未知的值或者一个值不存在。

包含`null`的任何算术表达式的结果为`null` z示例:`5 + null`返回`null`

“谓词为`null`”可用于检查空值。

z示例:查找所有工资为空的教员。选择的名字
从教练
其中`salary`为`null`



空值和三值逻辑

与 $null$ 的任何比较都返回未知

z 示例: $5 < null$ 或 $null < 5$ 或 $null = null$

使用真值 $unknown$ 的三值逻辑: z OR: ($unknown$ OR $true$) = $true$,

($unknown$ OR $false$) = $unknown$
(未知或未知)=未知

z AND: ($true$ AND $unknown$) = $unknown$,
($false$ AND $unknown$) = $false$,
(未知未知)=未知

z NOT: (非未知)=未知

z "P是未知的"求值为真, 如果谓词 P 求值为未知

如果 $where$ 子句谓词的计算结果为未知, 则将其结果视为假



聚合函数

☒ 这些函数对关系列的多值集进行操作，并返回一个值

Avg:平均值 **min:**最小值

max:最大值 **sum:**值的

总和 **count:**值的个数



Aggregate Functions (Cont.)

“查找计算机科学系讲师的平均工资

z 选择 avg(工资)

从教练

where dept_name = 'Comp. Sci.' ;

“查找2010年春季学期讲授某门课程的讲师总数

z **select count**(不同的ID)

从教

其中学期 = “春天”, 年 = 2010

“求课程关系中元组的个数

z **select count** (*)

从课程;



聚合函数-按分组

- 查找各部门教官的平均工资 `select dept_name, avg`
(salary)
从教练
组由 `dept_name`;

ID	name	dept_name	salary
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

dept_name	avg_salary
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000



聚合(续)。

聚集函数之外的**select**子句中的属性必须出现在**group by**列表中

z /*错误查询*/

```
select dept_name, ID, avg (salary)
```

从教练

组由dept_name;



聚合函数-Having子句

“查找平均工资大于42000的所有部门的名称和平均工资

从教员中选择`dept_name`, **avg**

(`salary`)

按 `dept_name` 分 组 , **avg**

(`salary`) > 42000;

注意:**having**子句中的谓词在组形成之后应用, 而**where**子句中的谓词在组形成之前应用



空值和聚合

☒ 所有工资总额

选择总和(工资)
从教练

上面的语句忽略空量

z 如果没有非空金额, Result为空

除**count(*)**外的所有聚合操作都会忽略聚合属性上具有空值的元组

“如果集合只有空值怎么办?”

z count返回0

z 所有其他聚合返回null



嵌套的子查询

SQL为子查询的嵌套提供了一种机制。

子查询是一个嵌套在另一个查询中的**select-from-where**表达式。

子查询的一个常见用途是执行集成员、集比较和集基数的测试。



示例查询

“查找2009年秋季和2010年春季提供的课程

选择不同的`course_id`

从部分

where `semester = ' Fall '` **and** `year= 2009` **and**

`course_id in` (select `course_id`

从部分

其中`semester = ' Spring '` **and** `year= 2010`);

“查找2009年秋季而不是2010年春季提供的课程。

选择不同的`course_id`

从部分

where `semester = ' Fall '` **and** `year= 2009` **and**

`course_id not in` (select `course_id`

从部分

其中`semester = ' Spring '` **and** `year= 2010`);



示例查询

“查找已参加ID为10101的讲师讲授的课程部分的(不同的)学生总数

```
select count(不同的ID)
从花
Where (course_id, sec_id, semester, year) in
      (select course_id, sec_id, semester, year
       from taught
        教的地方.ID = 10101);
```

注:上面的查询可以用更简单的方式编写。上面的提法只是为了说明SQL的特性。



Set Comparison

“查找工资高于生物系某些(至少一个)讲师的姓名。

选择不同的 $T.name$

from *instructor* **as** T , *instructor* **as** S

where $T.salary > S.salary$ **and** $S.dept\ name = 'Biology'$;

相同的查询，使用 $> some$ 子句

选择的 $名字$

从 $教练$

Where $salary > some$ (select salary

from *instructor* ?

where $dept\ name = 'Biology'$);



某些条款的定义

$F <comp> some\ r \Leftrightarrow \exists t \in r$ 使得 $(F <comp> t)$ 其中 $<comp>$ 可以是: $<, \leq, >, =, \neq$

(5 < 有的)

0
5
6

) = 真 (阅读: 5 < 关系中的某些元组)

(5 < **some**

05

) = 假

(5 = 一些)

0
5

) = 真

(5 $\neq$ 一些

0
5

) = true (因为 $0 \neq 5$)

(= **some**) $\equiv$ in 然而, ($\neq$ **some**) $\equiv$

not in

/



示例查询

“查找工资大于生物系所有教师工资的所有教师的姓名。

选择的~~名字~~

从教练

```
Where salary > all (select salary ?  
                        from instructor where dept  
                        name = ' biological ');
```



all Clause的定义

$F \text{ <comp> all } r \Leftrightarrow \forall t \in r (F \text{ <comp> } t)$

$(5 < \text{all } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline 6 \\ \hline \end{array}) = \text{false}$

$(5 < \text{all } \begin{array}{|c|} \hline 6 \\ \hline 10 \\ \hline \end{array}) = \text{true}$

$(5 = \text{all } \begin{array}{|c|} \hline 4 \\ \hline 5 \\ \hline \end{array}) = \text{false}$

$(5 \neq \text{all } \begin{array}{|c|} \hline 4 \\ \hline 6 \\ \hline \end{array}) = \text{true (since } 5 \neq 4 \text{ and } 5 \neq 6)$

$(\neq \text{all}) \equiv \text{not in}$

However, $(= \text{all}) \not\equiv \text{in}$



空关系的检验

如果参数子查询是非空的，**exists**构造返回值**true**。

exists $r \Leftrightarrow r \neq \emptyset$

“不存在 r ” $\Leftrightarrow r = \emptyset$



相关变量

另一种指定查询“查找2009年秋季学期和2010年春季学期教授的所有课程”的方法是**选择***course_id*

from *section* **as** *S*

where *semester* = ' Fall ' **and** *year* = 2009 **and**

exists (select * ?

from *section* **as** *T*

where *semester* = ' Spring ' **and** *year* = 2010

and *S.course_id* = *T.course_id*);

“相关子查询”

” 相关名称或相关变量



不存在

☒ 找到所有修过生物系所有课程的学生。

选择不同的 $S.ID$, $S.name$

from *student* **as** S

Where not exists ((**select** *course_id* ?

从课程

where *dept_name* = ' Biology ')

except

(**select** $T.course_id$

from *takes* **as** T

where $S.ID = T.ID$));

*注意 $X - Y = \emptyset \Leftrightarrow X \subseteq Y$

注:不能使用 = **all** 及其变体来编写此查询



Test for absent of Duplicate Tuples

☒ **唯一**构造测试子查询的结果中是否有重复元组。

☒ 查找2009年最多提供一次的所有课程，选择 *T.course_id*

```
from course as T  
where unique (select R.course_id ?  
                from section as R  
                where T.course_id = R.course_id  
                and R.year = 2009);
```



派生的关系

SQL允许在from子句中使用子查询表达式

“找出那些平均工资超过4.2万美元的院系教师的平均工资。”

选择dept_name, avg_salary

From (select dept_name, avg (salary) as avg_salary

From instructor

按dept_name分组)

其中avg_salary > 42000;

注意，我们不需要使用**having**从句

“上述查询的另一种写法

选择dept_name, avg_salary

From (select dept_name, avg (salary))

从教练

按dept_name分组)为dept_avg (dept_name, avg_salary)

其中avg_salary > 42000;



派生关系(续)

” 还有另一种写法:侧边从句

```
Select name, salary, avg_salary  
从教员I1, 横向(select avg(salary) as avg_salary  
from 教 员 I2 where I2.dept_name= I1.  
dept_name);
```



With子句

with子句提供了一种定义临时视图的方法，该视图的定义仅对出现**with**子句的查询可用。

“查找预算最大的所有部门。

将 *max_budget* (value) 设 为

(select **max(budget)**)

从部门)

选择预算

从*department, max_budget*

在部门。*Budget = max_budget.value;*



使用With子句的复杂查询

“查找总工资大于所有部门总工资平均值的所有部门

用 *dept_total* (*dept_name*, *value*) 作为

(select *dept_name*, **sum**(*salary*) from

instructor

按 *dept_name*, *dept_total_avg*

(*value*) 分组

(选择 **avg**(值)

从 *dept_total*)

选择 *dept_name*

From *dept_total*, *dept_total_avg*

dept_total 的地方。Value >= *dept_total_avg.value*;



标量子查询

选择 *dept_name*,
 (select **count** (*)
 从教练
 哪里 *department.dept_name = instructor.dept_name*) 作为 *num_*
 instructor
从部门;



修改数据库-删除

☒ 删除所有教官

从教练中删除

☒ 删除财务部门的所有讲师

从教官中删除

where *dept_name* = ' Finance ';

删除与位于沃森大楼的部门相关联的教师关系中的所有元组。

从教员中删除

在(select *dept name* ?

from *department* **where**
building = '沃森');



示例查询

☒ 删除所有工资低于教官平均工资的教官

从教官中删除

Where *salary* < (select avg (**salary**) from **instructor**);

z 问题:当我们从存款中删除元组时, 平均工资发生了变化

z SQL中使用的解决方案:

1. 首先, 计算**avg**工资并找到所有要删除的元组
2. 接下来, 删除上面找到的所有元组(不需要重新计算**avg**或重新测试元组)



修改Database - insert

- ❑ 向course添加一个新元组

插入course

```
values (' CS-437 ', '数据库系统', '比较。科学。”,4);
```

“或等价地”

在课程 (course_id, title, dept_name, 学分) 中插入值 (' CS-437 ', '数据库系统', '比较 '). 科学。”,4);

- ❑ 将tot_creds设置为null的新元组添加到student中

```
values (' 3003 ', ' Green ', ' Finance ', null);
```



修改Database - insert

将tot_creds设置为0的所有教师添加到学生关系中，插入到student
中

```
select ID, name, dept_name, 0  
从教练
```

“**select from where**语句在其任何操作之前被完全求值
结果被插入到关系中(否则查询如**insert into table1 select ***
from table1)
会导致问题)



数据库的修改-更新

工资超过10万美元的教师工资增加3%，其他所有教师工资增加5%

z 写两份更新声明:

更新教师

设置 $工资 = 工资 * 1.03$

其中 $工资 > 100000$;

更新教师

设置 $工资 = 工资 * 1.05$

其中 $salary \leq 100000$;

z 顺序很重要

z 用 **case** 语句可以做得更好(下一张幻灯片)



条件更新的Case语句

- ☒ 与之前相同的查询，但使用case语句更新指导

Set salary = case

当 $salary \leq 100000$ 则 $salary * 1.05$ 否 则

$salary * 1.03$

结束



使用标量子查询进行更新

重新计算和更新所有学生的tot_creds值更新学生S

Set tot_cred = (select sum(学分))

From take natural join course

其中S.ID=取.ID和takes.grade

<> 'F', take. Grade不为空);

“对于没有上过任何课程的学生，将tot_creds设置为null” 代替**sum(学分)**，使用：

情况下

当**sum(学分)**不为空时，**sum(学分)**为0

结束



第三章结束

数据库系统概念，6thEd。

©Silberschatz, Korth和Sudarshan参见www.db-book.com了解
重用条件



高级SQL功能**

- ❑ 创建一个与现有表模式相同的表: **创建表** *temp_account* **like** *account*



Figure 3.02

<i>name</i>
Srinivasan
Wu
Mozart
Einstein
El Said
Gold
Katz
Califieri
Singh
Crick
Brandt
Kim



Figure 3.03

<i>dept_name</i>
Comp. Sci.
Finance
Music
Physics
History
Physics
Comp. Sci.
History
Finance
Biology
Comp. Sci.
Elec. Eng.



图3.04

<i>name</i>
Katz
Brandt



Figure 3.05

<i>name</i>	<i>dept_name</i>	<i>building</i>
Srinivasan	Comp. Sci.	Taylor
Wu	Finance	Painter
Mozart	Music	Packard
Einstein	Physics	Watson
El Said	History	Painter
Gold	Physics	Watson
Katz	Comp. Sci.	Taylor
Califieri	History	Painter
Singh	Finance	Painter
Crick	Biology	Watson
Brandt	Comp. Sci.	Taylor
Kim	Elec. Eng.	Taylor



Figure 3.06

<i>Inst.ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	<i>teaches.ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Physics	95000	10101	CS-101	1	Fall	2009
10101	Srinivasan	Physics	95000	10101	CS-315	1	Spring	2010
10101	Srinivasan	Physics	95000	10101	CS-347	1	Fall	2009
10101	Srinivasan	Physics	95000	10101	FIN-201	1	Spring	2010
10101	Srinivasan	Physics	95000	15151	MU-199	1	Spring	2010
10101	Srinivasan	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Physics	95000	10101	CS-101	1	Fall	2009
12121	Wu	Physics	95000	10101	CS-315	1	Spring	2010
12121	Wu	Physics	95000	10101	CS-347	1	Fall	2009
12121	Wu	Physics	95000	10101	FIN-201	1	Spring	2010
12121	Wu	Physics	95000	15151	MU-199	1	Spring	2010
12121	Wu	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
15151	Mozart	Physics	95000	10101	CS-101	1	Fall	2009
15151	Mozart	Physics	95000	10101	CS-315	1	Spring	2010
15151	Mozart	Physics	95000	10101	CS-347	1	Fall	2009
15151	Mozart	Physics	95000	10101	FIN-201	1	Spring	2010
15151	Mozart	Physics	95000	15151	MU-199	1	Spring	2010
15151	Mozart	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2009
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2010
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2009
22222	Einstein	Physics	95000	10101	FIN-201	1	Spring	2010
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...



Figure 3.07

<i>name</i>	<i>Course_id</i>
Srinivasan	CS-101
Srinivasan	CS-315
Srinivasan	CS-347
Wu	FIN-201
Mozart	MU-199
Einstein	PHY-101
El Said	HIS-351
Katz	CS-101
Katz	CS-319
Crick	BIO-101
Crick	BIO-301
Brandt	CS-190
Brandt	CS-190
Brandt	CS-319
Kim	EE-181



Figure 3.08

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Comp. Sci.	65000	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	CS-347	1	Fall	2009
12121	Wu	Finance	90000	FIN-201	1	Spring	2010
15151	Mozart	Music	40000	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	PHY-101	1	Fall	2009
32343	El Said	History	60000	HIS-351	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-319	1	Spring	2010
76766	Crick	Biology	72000	BIO-101	1	Summer	2009
76766	Crick	Biology	72000	BIO-301	1	Summer	2010
83821	Brandt	Comp. Sci.	92000	CS-190	1	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-190	2	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-319	2	Spring	2010
98345	Kim	Elec. Eng.	80000	EE-181	1	Spring	2009



Figure 3.09

<i>course_id</i>
CS-101
CS-347
PHY-101



Figure 3.10

<i>course_id</i>
CS-101
CS-315
CS-319
CS-319
FIN-201
HIS-351
MU-199



Figure 3.11

<i>course_id</i>
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101



图3.12

<i>course_id</i>
CS-101



图3.13

<i>course_id</i>
CS-347
PHY-101



Figure 3.14

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000



Figure 3.15

<i>dept_name</i>	<i>salary</i>
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000



Figure 3.16

<i>dept_name</i>	<i>count</i>
Comp. Sci.	3
Finance	1
History	1
Music	1



Figure 3.17

<i>dept_name</i>	<i>avg(salary)</i>
Physics	91000
Elec. Eng.	80000
Finance	85000
Comp. Sci.	77333
Biology	72000
History	61000



数据定义语言

不仅允许规范一组关系，还允许规范每个关系的信息，包括：

每个关系的模式。

与每个属性相关联的值的域。完整性约束

要为每个关系维护的索引集。每个关系的安全和授权信息。每个关系在磁盘上的物理存储结构。